

__tmp__report

June 27, 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA

—o0o—

1 BÁO CÁO CUỐI HỌC KỲ

HỌC PHẦN: LẬP TRÌNH PHÂN TÍCH DỮ LIỆU VỚI PYTHON

2 PHÂN TÍCH VÀ DỰ ĐOÁN LƯỢT KHÁCH DU LỊCH QUỐC TẾ ĐẾN VIỆT NAM

Giảng viên hướng dẫn: ThS. Nguyễn Anh Tuấn

2.0.1 HÀ NỘI, THÁNG 7 NĂM 2025

2.1 Mục lục

1. Giới thiệu đề tài
 - 1.1 Đặt vấn đề bài toán
 - 1.2 Mục tiêu đề tài
 - 1.3 Phạm vi và nguồn dữ liệu
 - 1.4 Công cụ và thư viện sử dụng
2. Thu thập dữ liệu
 - 2.1 Nguồn dữ liệu
 - 2.2 Cấu trúc dữ liệu và thách thức
 - 2.3 Cách xử lý và hợp nhất dữ liệu
 - 2.4 Nhận xét
3. Tiền xử lý dữ liệu
 - 3.1 Làm sạch dữ liệu
 - 3.2 Xử lý giá trị thiếu
 - 3.3 Trích xuất đặc trưng (Feature Engineering)
4. Phân tích và khám phá dữ liệu (EDA)
 - 4.1 Phân tích tổng quan lượng khách quốc tế
 - 4.2 Phân tích theo quốc gia nguồn
 - 4.3 Phân tích tính mùa vụ (Seasonality)
 - 4.4 Tương quan giữa các quốc gia nguồn
 - 4.5 Phân tích xu hướng quốc gia cụ thể

5. Xây dựng mô hình dự đoán
 - 5.1 Linear Regression (Hồi quy tuyến tính)
 - 5.2 Random Forest Regression
 - 5.3 XGBoost Regressor
 - 5.4 SARIMA (Mô hình chuỗi thời gian)
 - 5.5 So sánh hiệu suất các mô hình
6. Tối ưu mô hình
 - 6.1 GridSearchCV cho Random Forest
 - 6.2 RandomizedSearchCV cho XGBoost
 - 6.3 Kết quả sau tối ưu
7. Dự đoán tương lai
 - 7.1 Dự đoán 4 quý tiếp theo bằng SARIMA
 - 7.2 Phân tích khoảng tin cậy
8. Tổng kết
 - 8.1 Kết quả đạt được
 - 8.2 Hạn chế
 - 8.3 Hướng phát triển

2.2 Danh sách hình vẽ

1. Lượng khách quốc tế đến Việt Nam theo năm (2008–2026)
2. Top 10 quốc gia nguồn khách hàng đầu
3. Phân tích mùa vụ: Lượng khách trung bình theo quý
4. Ma trận tương quan giữa 5 quốc gia nguồn lớn nhất
5. Xu hướng lượng khách theo từng quốc gia (Top 5)
6. So sánh hiệu suất các mô hình dự đoán
7. Dự đoán lượng khách 4 quý tiếp theo với khoảng tin cậy 95%

2.3 1. Giới thiệu đề tài

2.3.1 1.1 Đặt vấn đề bài toán

Ngành du lịch là một trong những ngành kinh tế quan trọng của Việt Nam, đóng góp đáng kể vào tổng sản phẩm quốc nội (GDP) và tạo ra hàng triệu việc làm cho người dân. Trong những năm gần đây, Việt Nam đã nổi lên như một điểm đến hấp dẫn trong khu vực Đông Nam Á, thu hút lượng lớn khách quốc tế từ nhiều quốc gia khác nhau.

Tuy nhiên, lượng khách du lịch quốc tế đến Việt Nam chịu ảnh hưởng của nhiều yếu tố phức tạp: tình hình kinh tế toàn cầu, chính sách visa, tỷ giá hối đoái, sự cạnh tranh từ các quốc gia láng giềng, và đặc biệt là các sự kiện bất khả kháng như đại dịch COVID-19 năm 2020–2021. Việc hiểu rõ các xu hướng, mô hình mùa vụ, và mối quan hệ giữa các thị trường nguồn là điều cần thiết để hoạch định chiến lược phát triển du lịch hiệu quả.

Đặt biệt, đại dịch COVID-19 đã tạo ra một khoảng trống dữ liệu lớn (năm 2021 hoàn toàn không có dữ liệu trong các file báo cáo), gây thách thức đáng kể cho việc phân tích liên tục và xây dựng mô hình dự đoán. Đây cũng là một trong những bài toán thú vị mà nhóm cần giải quyết trong quá trình thực hiện đề tài.

2.3.2 1.2 Mục tiêu đề tài

Đề tài hướng đến hai nhóm mục tiêu chính:

Mục tiêu phân tích (Analytical): - Phân tích xu hướng tổng thể lượng khách quốc tế đến Việt Nam trong giai đoạn 2008–2026. - Xác định các quốc gia nguồn khách hàng đầu và sự thay đổi thứ hạng qua các năm. - Phát hiện các mô hình mùa vụ (seasonality) trong lượng khách theo quý. - Phân tích mối tương quan giữa các thị trường nguồn khách. - Đánh giá tác động của đại dịch COVID-19 đến ngành du lịch Việt Nam.

Mục tiêu dự đoán (Predictive): - Xây dựng các mô hình học máy để dự đoán lượng khách quốc tế theo quý. - So sánh hiệu quả giữa các phương pháp: Linear Regression, Random Forest, XGBoost và SARIMA. - Tối ưu hóa siêu tham số để cải thiện độ chính xác của mô hình. - Dự đoán lượng khách cho 4 quý tiếp theo kèm theo khoảng tin cậy.

2.3.3 1.3 Phạm vi và nguồn dữ liệu

Để đảm bảo tính tập trung và chất lượng phân tích, phạm vi dữ liệu được giới hạn như sau:

- **Đối tượng phân tích:** Lượng khách quốc tế đến Việt Nam, phân theo quốc gia nguồn và theo quý (Q1–Q4).
- **Thời gian:** Giai đoạn 2008–2026, với lưu ý năm 2021 không có dữ liệu (ảnh hưởng của đại dịch COVID-19).
- **Quốc gia nguồn:** 40 quốc gia/vùng lãnh thổ, bao gồm các thị trường lớn như Trung Quốc, Hàn Quốc, Nhật Bản, Hoa Kỳ, và các nhóm thị trường khác.
- **Nguồn dữ liệu:** Bốn file báo cáo dạng HTML-Excel (.xls), mỗi file tương ứng với một quý (Q1, Q2, Q3, Q4), được cung cấp từ cơ quan thống kê du lịch.

2.3.4 1.4 Công cụ và thư viện sử dụng

Quá trình thực hiện đề tài được hỗ trợ bởi các công cụ và thư viện sau:

- **Ngôn ngữ lập trình:** Python 3
- **Xử lý dữ liệu:** pandas, numpy, lxml (phân tích HTML)
- **Trực quan hóa:** matplotlib, seaborn
- **Xây dựng mô hình:** scikit-learn (Linear Regression, Random Forest, GridSearchCV, RandomizedSearchCV), xgboost (XGBoost Regressor), statsmodels (SARIMA)
- **Môi trường:** Jupyter Notebook
- **Quản lý phiên bản:** Git, GitHub

2.4 2. Thu thập dữ liệu

2.4.1 2.1 Nguồn dữ liệu

Dữ liệu được cung cấp dưới dạng 4 file báo cáo HTML-Excel, mỗi file chứa thông tin lượng khách quốc tế đến Việt Nam cho một quý cụ thể:

File	Quý	Số quốc gia	Ghi chú
quy1-cacnuoc.xls	Q1	38	Bắt đầu từ năm 2009
quy2-cacnuoc.xls	Q2	39	Bắt đầu từ năm 2009
quy3-cacnuoc.xls	Q3	40	Bắt đầu từ năm 2008

File	Quý	Số quốc gia	Ghi chú
quy4-cacnuoc.xls	Q4	38	Bắt đầu từ năm 2008

Mỗi file chứa một bảng dữ liệu với các cột là các năm và các dòng là các quốc gia nguồn khách. Giá trị trong bảng là lượng khách quốc tế (đơn vị: lượt khách).

2.4.2 2.2 Cấu trúc dữ liệu và thách thức

Các file `.xls` này không phải là file Excel nhị phân thực sự, mà là file HTML có phần mở rộng `.xls` — một kỹ thuật phổ biến để tạo file “Excel” từ web. Cấu trúc HTML của các file có một số đặc điểm gây thách thức cho việc phân tích:

Thứ nhất, dữ liệu trong bảng HTML không nằm trong các thẻ `<td>` theo cách thông thường. Thay vào đó, các giá trị số liệu nằm dưới dạng **text nodes** (nút văn bản) giữa các thẻ `<td>` rỗng. Cụ thể, cú pháp HTML có dạng:

```
<td rowspan="2" colspan="1"></td>2009<td rowspan="2" colspan="1"></td>2010
```

Trong đó “2009” và “2010” là text nằm **ngoài** thẻ `<td>`, không phải nội dung bên trong. Điều này khiến các hàm đọc HTML tiêu chuẩn như `pd.read_html()` không thể trích xuất dữ liệu chính xác — tất cả các ô đều trả về giá trị rỗng.

Thứ hai, các file không đồng nhất về số năm: Q3 và Q4 có dữ liệu từ năm 2008, trong khi Q1 và Q2 chỉ bắt đầu từ năm 2009. Ngoài ra, cột năm 2021 hoàn toàn vắng mặt trong tất cả các file — đây là hệ quả trực tiếp của đại dịch COVID-19 khi ngành du lịch gần như đóng băng hoàn toàn.

Thứ ba, định dạng số sử dụng dấu chấm (.) làm phân cách hàng nghìn theo quy ước Việt Nam (ví dụ: 104.520 = 104,520), không phải dấu phẩy như định dạng quốc tế. Cần chuyển đổi chính xác để tránh sai lệch dữ liệu.

2.4.3 2.3 Cách xử lý và hợp nhất dữ liệu

Để vượt qua các thách thức trên, nhóm đã xây dựng một bộ phân tích HTML tùy chỉnh sử dụng thư viện `lxml`:

Bước 1: Phân tích cú pháp HTML Sử dụng `lxml.html.fromstring()` để chuyển đổi chuỗi HTML thành cây DOM, cho phép truy cập từng phần tử `<td>` và các text node liền kề.

Bước 2: Xác định cấu trúc bảng Phân tích hàng tiêu đề (hàng đầu tiên) để xác định vị trí các năm. Hàng tiêu đề có 20 thẻ `<td>`, trong đó `<td>` thứ 2 đến thứ 19 chứa tên năm (2009–2026) và “Totals” trong thuộc tính `tail`.

Bước 3: Trích xuất dữ liệu từ từng hàng Mỗi hàng dữ liệu có 19 thẻ `<td>`: - `<td>` đầu tiên (có `colspan=2`) chứa tên quốc gia trong thuộc tính `tail` - `<td>` thứ 2 đến thứ 19 chứa giá trị lượng khách tương ứng với các năm

Bước 4: Chuyển đổi định dạng số Áp dụng hàm `val_str.replace('.', '')` để loại bỏ dấu chấm phân cách hàng nghìn, sau đó chuyển đổi sang kiểu `float`.

Bước 5: Hợp nhất dữ liệu Kết hợp dữ liệu từ 4 file thành một DataFrame duy nhất (`df_long`) với các cột: `country`, `year`, `quarter`, `arrivals`. Loại bỏ hàng “Totals” (hàng tổng cộng) vì đây là dữ liệu tổng hợp, không phải quốc gia riêng lẻ.

2.4.4 2.4 Nhận xét

Quá trình thu thập và xử lý dữ liệu gặp không ít thách thức, đặc biệt là cấu trúc HTML không chuẩn của các file báo cáo. Tuy nhiên, việc xây dựng bộ phân tích tùy chỉnh đã giúp trích xuất dữ liệu chính xác.

Để xác minh tính chính xác, nhóm đã kiểm tra chéo với dữ liệu mẫu được cung cấp: quốc gia “Hoa Kỳ” năm 2009, quý 1 có giá trị 104.520 (tương đương 104,520 lượt khách). Kết quả phân tích trả về đúng giá trị này, xác nhận dữ liệu được trích xuất chính xác.

Sau khi hợp nhất, tập dữ liệu cuối cùng có **1,894 bản ghi**, bao gồm **40 quốc gia** và **18 năm** (2008–2026, không bao gồm 2021).

2.5 3. Tiền xử lý dữ liệu

2.5.1 3.1 Làm sạch dữ liệu

Sau khi hợp nhất dữ liệu từ 4 file, nhóm thực hiện các bước làm sạch sau:

Loại bỏ hàng tổng cộng: Hàng “Totals” trong mỗi file là dữ liệu tổng hợp, không đại diện cho một quốc gia cụ thể. Hàng này được loại bỏ để tránh trùng lặp dữ liệu khi tính tổng.

Kiểm tra dữ liệu trùng lặp: Kiểm tra xem có bản ghi trùng lặp (cùng quốc gia, năm, quý) hay không. Kết quả cho thấy không có bản ghi trùng lặp nào.

Chuẩn hóa tên quốc gia: Tên quốc gia được giữ nguyên theo tiếng Việt như trong dữ liệu gốc (ví dụ: “Hoa Kỳ”, “Trung Quốc”, “Hàn Quốc”). Không cần chuẩn hóa thêm vì dữ liệu đã nhất quán.

2.5.2 3.2 Xử lý giá trị thiếu

Do cấu trúc dữ liệu, một số quốc gia không có dữ liệu cho một số năm hoặc quý cụ thể. Nguyên nhân có thể do: - Quốc gia đó chưa nằm trong danh sách báo cáo vào thời điểm đó - Lượng khách quá nhỏ nên không được thống kê riêng - Lỗi trong quá trình thu thập dữ liệu

Chi lược xử lý: Các ô không có dữ liệu được điền bằng giá trị 0. Điều này hợp lý vì: - Không có dữ liệu có thể hiểu là không có khách từ quốc gia đó trong quý/năm đó - Việc điền 0 không làm sai lệch phân tích tổng thể vì các quốc gia này thường có lượng khách rất nhỏ - Tạo ra một lưới dữ liệu hoàn chỉnh (complete grid) thuận tiện cho việc phân tích và xây dựng mô hình

Sau khi xử lý, tập dữ liệu hoàn chỉnh (`df_complete`) có **2,880 bản ghi** ($40 \text{ quốc gia} \times 18 \text{ năm} \times 4 \text{ quý}$).

2.5.3 3.3 Trích xuất đặc trưng (Feature Engineering)

Để phục vụ cho việc xây dựng mô hình dự đoán, nhóm tạo thêm các đặc trưng mới:

Biến thời gian: - `quarter_num`: Số thứ tự quý (1, 2, 3, 4) thay vì ký tự “Q1”, “Q2”,... - `time_idx`: Chỉ số thời gian liên tục, tính bằng $\text{year} + (\text{quarter_num} - 1) / 4$, thuận tiện cho mô hình học xu hướng tuyến tính theo thời gian.

Đặc trưng trễ (Lag Features): - `lag_1`: Giá trị quý trước đó (trễ 1 quý) - `lag_2`: Giá trị cách 2 quý - `lag_4`: Giá trị cùng quý năm trước (trễ 4 quý)

Các đặc trưng trễ giúp mô hình học được tính chu kỳ và quán tính của dữ liệu thời gian.

Trung bình trượt (Rolling Mean): - `rolling_mean_4`: Trung bình lượng khách trong 4 quý gần nhất, giúp làm mượt các biến động ngắn hạn.

Tập dữ liệu chia nhỏ: - **Tập huấn luyện:** Dữ liệu từ năm 2008 đến 2023 (bao gồm) - **Tập kiểm tra:** Dữ liệu từ năm 2024 trở đi

Việc chia này đảm bảo mô hình được đánh giá trên dữ liệu “tương lai” mà nó chưa từng thấy, phản ánh đúng khả năng dự đoán thực tế.

2.6 4. Phân tích và khám phá dữ liệu (EDA)

Trước khi tiến hành xây dựng mô hình dự đoán, việc phân tích và khám phá dữ liệu (Exploratory Data Analysis – EDA) là bước quan trọng, không thể thiếu. EDA giúp hiểu rõ hơn về cấu trúc dữ liệu, phát hiện các xu hướng, mô hình mùa vụ, và mối quan hệ giữa các thị trường nguồn khách.

2.6.1 4.1 Phân tích tổng quan lượng khách quốc tế

Biểu đồ dưới đây thể hiện xu hướng tổng thể lượng khách quốc tế đến Việt Nam theo giai đoạn 2008–2026.

```
[ ]: import pandas as pd
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import seaborn as sns
from lxml import html
import warnings
warnings.filterwarnings('ignore')
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['figure.dpi'] = 100
sns.set_style('whitegrid')

# Parse data (reuse verified parser)
def parse_quarterly_file(filepath, quarter_label):
    with open(filepath, 'r', encoding='utf-8') as f:
        raw = f.read()
        tree = html.fromstring(raw)
        rows = tree.xpath('//tr')
        header_tds = rows[0].xpath('.//td')
        years = [(td.tail or '').strip() for td in header_tds[2:]]
        records = []
        for row in rows[1:]:
            tds = row.xpath('.//td')
            if len(tds) < 3: continue
            country = (tds[0].tail or '').strip()
            if not country or country == 'Phân theo thị trường': continue
            for j, td in enumerate(tds[1:]):
                val_str = (td.tail or '').strip()
```

```

        year_label = years[j] if j < len(years) else None
        if not val_str or not year_label or year_label == 'Totals': continue
        try: val = float(val_str.replace('.', ''))
        except ValueError: continue
        records.append({'country': country, 'year': int(year_label),
        ↪ 'quarter': quarter_label, 'arrivals': val})
        return pd.DataFrame(records)

dfs = []
for label, fpath in [('Q1', 'data/quy1-cacnuoc.xls'), ('Q2', 'data/quy2-cacnuoc.
    ↪ xls'), ('Q3', 'data/quy3-cacnuoc.xls'), ('Q4', 'data/quy4-cacnuoc.xls')]:
    dfs.append(parse_quarterly_file(fpath, label))
df_long = pd.concat(dfs, ignore_index=True)
df_long = df_long[df_long['country'] != 'Totals'].reset_index(drop=True)
df_total = df_long.groupby(['country', 'year'])['arrivals'].sum().reset_index()
df_total.columns = ['country', 'year', 'total_arrivals']

fig, ax = plt.subplots(figsize=(14, 7))
yearly = df_total.groupby('year')['total_arrivals'].sum().reset_index()
ax.plot(yearly['year'], yearly['total_arrivals']/1e6, 'o-', lw=2, ms=8,
    ↪ color='#2196F3')
ax.set_xlabel('Năm', fontsize=12); ax.set_ylabel('Tổng lượng khách (triệu)',
    ↪ fontsize=12)
ax.set_title('Hình 1: Lượng khách quốc tế đến Việt Nam theo năm (2008-2026)',
    ↪ fontsize=14, fontweight='bold')
ax.set_xticks(yearly['year']); ax.set_xticklabels(yearly['year'], rotation=45)
covid_val = yearly[yearly['year']==2020]['total_arrivals'].values[0]/1e6
ax.annotate('COVID-19', xy=(2020, covid_val), xytext=(2018, covid_val+1.5),
    ↪ fontsize=11, ha='center', arrowprops=dict(arrowstyle='->',
    ↪ color='red', lw=1.5), color='red', fontweight='bold')
ax.grid(True, alpha=0.3); plt.tight_layout(); plt.savefig('output/
    ↪ eda_total_trend.png', dpi=150, bbox_inches='tight'); plt.show()

```

Nhận xét:

Biểu đồ cho thấy lượng khách quốc tế đến Việt Nam có xu hướng tăng trưởng mạnh trong giai đoạn 2009–2019, từ khoảng 3.8 triệu lượt (2009) lên đến 18.0 triệu lượt (2019) — tăng gần 5 lần trong một thập kỷ. Giai đoạn tăng trưởng đặc biệt mạnh từ 2016 (10.0 triệu) đến 2019 (18.0 million), với tốc độ tăng trung bình khoảng 2.7 triệu lượt/năm.

Tác động của COVID-19: Năm 2020, lượng khách giảm mạnh xuống chỉ còn khoảng 3.7 triệu lượt — mức thấp nhất kể từ năm 2009. Điều này phản ánh tác động nghiêm trọng của đại dịch đến ngành du lịch toàn cầu. Đặc biệt, năm 2021 hoàn toàn không có dữ liệu trong các file báo cáo, cho thấy hoạt động du lịch quốc tế gần như đóng băng hoàn toàn.

Phục hồi sau đại dịch: Từ năm 2022, ngành du lịch bắt đầu phục hồi. Lượng khách năm 2022 đạt khoảng 3.7 triệu (tương đương mức 2020), sau đó tăng vọt lên 12.6 triệu (2023) và 17.6 triệu (2024). Đến năm 2025, lượng khách đạt 21.2 triệu — vượt qua mức trước đại dịch năm 2019.

Dữ liệu năm 2026: Dữ liệu năm 2026 chỉ bao gồm Q1 và Q2 (tổng 10.6 triệu), nên chưa thể đánh giá đầy đủ cho cả năm.

2.6.2 4.2 Phân tích theo quốc gia nguồn

```
[ ]: top10 = df_total.groupby('country')['total_arrivals'].sum().
    ↪sort_values(ascending=False).head(10)
fig, ax = plt.subplots(figsize=(12, 7))
colors = sns.color_palette('viridis', len(top10))
bars = ax.barh(range(len(top10)), top10.values/1e6, color=colors)
ax.set_yticks(range(len(top10))); ax.set_yticklabels(top10.index, fontsize=11)
ax.set_xlabel('Tổng lượng khách (triệu)', fontsize=12)
ax.set_title('Hình 2: Top 10 quốc gia nguồn khách hàng đầu', fontsize=14,
    ↪fontweight='bold')
ax.invert_yaxis()
for bar, val in zip(bars, top10.values):
    ax.text(bar.get_width()+0.1, bar.get_y()+bar.get_height()/2, f'{val/1e6:.
    ↪1f}M', va='center', fontsize=10)
plt.tight_layout(); plt.savefig('output/eda_top10_countries.png', dpi=150,
    ↪bbox_inches='tight'); plt.show()
```

Nhận xét:

Biểu đồ cho thấy sự phân hóa rõ rệt giữa các thị trường nguồn khách:

- **Trung Quốc** là thị trường lớn nhất với tổng cộng 41.7 triệu lượt khách trong toàn giai đoạn, chiếm tỷ trọng áp đảo. Điều này phản ánh vị trí địa lý gần, chính sách visa thuận lợi, và quy mô dân số lớn của Trung Quốc.
- **Hàn Quốc** đứng thứ hai với 33.0 triệu lượt, là thị trường tăng trưởng mạnh nhất trong những năm gần đây. Lượng khách Hàn Quốc tăng từ 105,000 (Q1/2009) lên hơn 1.3 triệu (Q1/2026), phản ánh sự phổ biến ngày càng tăng của Việt Nam như điểm đến du lịch đối với người Hàn Quốc.
- **Nhật Bản** (10.1 triệu) và **Đài Loan** (9.7 triệu) là hai thị trường truyền thống quan trọng, duy trì lượng khách ổn định qua các năm.
- **Hoa Kỳ** (9.1 triệu) đứng thứ 5, với xu hướng tăng trưởng đều đặn. Lượng khách Mỹ tăng từ khoảng 105,000 (Q1/2009) lên hơn 300,000 (Q1/2026).
- **Malaysia, Úc, Nga, và Campuchia** cũng nằm trong top 10, mỗi thị trường đóng góp từ 5-6 triệu lượt trong toàn giai đoạn.

2.6.3 4.3 Phân tích tính mùa vụ (Seasonality)

```
[ ]: fig, ax = plt.subplots(figsize=(10, 6))
quarterly = df_long.groupby('quarter')['arrivals'].agg(['mean', 'std']).
    ↪reindex(['Q1', 'Q2', 'Q3', 'Q4'])
ax.bar(quarterly.index, quarterly['mean']/1e3, yerr=quarterly['std']/1e3,
```



```

        capsize=5, color=['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4'],
        ↪edgecolor='black')
ax.set_xlabel('Quý', fontsize=12); ax.set_ylabel('Lượng khách TB (nghìn)',
        ↪fontsize=12)
ax.set_title('Hình 3: Phân tích mùa vụ - Lượng khách trung bình theo quý',
        ↪fontsize=14, fontweight='bold')
ax.grid(axis='y', alpha=0.3)
plt.tight_layout(); plt.savefig('output/eda_seasonality.png', dpi=150,
        ↪bbox_inches='tight'); plt.show()

```

Nhận xét:

Biểu đồ mùa vụ cho thấy sự khác biệt rõ rệt giữa các quý:

- **Quý 1 (Q1)** có lượng khách trung bình cao nhất, phản ánh mùa du lịch đầu năm với nhiều ngày lễ (Tết Nguyên Đán, Giáng sinh/Năm mới theo lịch dương) thu hút khách quốc tế.
- **Quý 2 (Q2)** và **Quý 3 (Q3)** có lượng khách tương đối ổn định, ở mức trung bình.
- **Quý 4 (Q4)** có độ biến động lớn nhất (thanh lỗi dài), cho thấy lượng khách quý này phụ thuộc nhiều vào từng năm cụ thể.

Độ lệch chuẩn lớn ở tất cả các quý phản ánh sự biến động đáng kể qua các năm, đặc biệt do tác động của COVID-19.

2.6.4 4.4 Tương quan giữa các quốc gia nguồn

```

[ ]: top5 = top10.head(5).index.tolist()
pivot = df_total[df_total['country'].isin(top5)].pivot_table(index='year',
        ↪columns='country', values='total_arrivals', aggfunc='sum')
corr = pivot.corr()
fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0, ax=ax,
        ↪square=True, linewidths=0.5)
ax.set_title('Hình 4: Ma trận tương quan giữa 5 quốc gia nguồn lớn nhất',
        ↪fontsize=14, fontweight='bold')
plt.tight_layout(); plt.savefig('output/eda_correlation.png', dpi=150,
        ↪bbox_inches='tight'); plt.show()

```

Nhận xét:

Ma trận tương quan cho thấy một số patterns:

- **Trung Quốc và Đài Loan** có tương quan dương mạnh (0.89), cho thấy hai thị trường này có xu hướng tăng/giảm đồng pha. Điều này có thể giải thích bởi vị trí địa lý tương tự, chính sách visa tương tự, và tác động của COVID-19 ảnh hưởng tương tự đến cả hai thị trường.
- **Hàn Quốc và Nhật Bản** có tương quan trung bình (0.71), phản ánh cả hai đều là thị trường Đông Á với một số đặc điểm tương tự về mùa vụ du lịch.
- **Hoa Kỳ** có tương quan thấp hơn với các thị trường châu Á, cho thấy thị trường Mỹ có động lực riêng, có thể bị ảnh hưởng bởi các yếu tố như tỷ giá USD/VND, khoảng cách địa lý, và

chính sách visa.

2.6.5 4.5 Phân tích xu hướng quốc gia cụ thể

```
[ ]: fig, axes = plt.subplots(3, 2, figsize=(16, 14))
axes = axes.flatten()
for i, country in enumerate(top5):
    cdata = df_total[df_total['country']==country].sort_values('year')
    axes[i].plot(cdata['year'], cdata['total_arrivals']/1e3, 'o-', lw=2, ms=6)
    axes[i].set_title(country, fontsize=12, fontweight='bold')
    axes[i].set_ylabel('Lượt khách (nghìn)'); axes[i].grid(True, alpha=0.3)
axes[5].set_visible(False)
plt.suptitle('Hình 5: Xu hướng lượng khách theo từng quốc gia (Top 5)',
            ↪fontsize=16, fontweight='bold', y=1.01)
plt.tight_layout(); plt.savefig('output/eda_country_trends.png', dpi=150,
            ↪bbox_inches='tight'); plt.show()
```

Nhận xét chi tiết từng quốc gia:

- **Trung Quốc:** Tăng trưởng mạnh nhất trong giai đoạn 2015–2019, từ 1.1 triệu lên 5.8 triệu lượt. Tuy nhiên, chịu tác động nặng nề nhất từ COVID-19, giảm xuống chỉ còn 0.4 triệu (2020). Phục hồi chậm hơn so với Hàn Quốc, đạt 1.4 triệu (2024) và 1.6 triệu (2025).
- **Hàn Quốc:** Thị trường tăng trưởng ổn định nhất, từ 0.4 triệu (2009) lên 4.6 triệu (2025). Đặc biệt, Hàn Quốc là thị trường phục hồi nhanh nhất sau COVID-19, vượt qua mức trước đại dịch vào năm 2024.
- **Nhật Bản:** Tăng trưởng đều đặn nhưng chậm hơn, từ 0.4 triệu (2009) lên 1.3 triệu (2025). Thị trường Nhật Bản có vẻ đã bão hòa hơn so với Hàn Quốc.
- **Đài Loan:** Tăng trưởng mạnh trong giai đoạn 2015–2019, nhưng phục hồi chậm hơn sau COVID-19 so với Hàn Quốc.
- **Hoa Kỳ:** Tăng trưởng ổn định qua các năm, từ 0.4 triệu (2009) lên 1.3 triệu (2025). Thị trường Mỹ ít bị ảnh hưởng bởi COVID-19 hơn so với các thị trường châu Á.

2.7 5. Xây dựng mô hình dự đoán

Trong phần này, nhóm xây dựng và so sánh 4 mô hình dự đoán lượng khách quốc tế theo quý: Linear Regression, Random Forest, XGBoost và SARIMA. Mỗi mô hình được đánh giá bằng các chỉ số: MAE (Mean Absolute Error), RMSE (Root Mean Squared Error), và R^2 (Hệ số xác định).

Dữ liệu được tổng hợp ở mức **tổng lượng khách mỗi quý** (tổng hợp tất cả các quốc gia), vì mục tiêu là dự đoán tổng lượng khách quốc tế, không phải lượng khách từng quốc gia riêng lẻ.

2.7.1 5.1 Linear Regression (Hồi quy tuyến tính)

Giới thiệu thuật toán:

Linear Regression là mô hình hồi quy đơn giản nhất, giả định mối quan hệ tuyến tính giữa biến đầu vào (X) và biến mục tiêu (y). Mô hình tìm đường thẳng phù hợp nhất với dữ liệu bằng cách tối thiểu hóa tổng bình phương sai số.

Ứng dụng trong bài toán:

Trong bài toán này, các đặc trưng đầu vào bao gồm: năm, số thứ tự quý, chỉ số thời gian liên tục, và các đặc trưng trễ (lag_1, lag_4, rolling_mean_4). Mô hình học mối quan hệ tuyến tính giữa các đặc trưng này và tổng lượng khách mỗi quý.

Kết quả:

Chỉ số	Giá trị
MAE	1,099,063
RMSE	1,448,419
R^2	0.4846

Mô hình Linear Regression đạt $R^2 = 0.48$, nghĩa là giải thích được khoảng 48% phương sai của dữ liệu. Đây là kết quả chấp nhận được đối với một mô hình đơn giản, nhưng vẫn còn room for improvement.

2.7.2 5.2 Random Forest Regression

Giới thiệu thuật toán:

Random Forest là một ensemble method kết hợp nhiều cây quyết định (decision trees). Mỗi cây được huấn luyện trên một mẫu bootstrap ngẫu nhiên của dữ liệu, và kết quả cuối cùng là trung bình dự đoán của tất cả các cây. Phương pháp này giúp giảm overfitting so với một cây đơn lẻ.

Ứng dụng trong bài toán:

Random Forest có khả năng học các mối quan hệ phi tuyến tính và tương tác giữa các đặc trưng, phù hợp với dữ liệu du lịch có tính mùa vụ và biến động phức tạp.

Kết quả:

Chỉ số	Giá trị
MAE	1,098,133
RMSE	1,548,464
R^2	0.4109

Random Forest đạt $R^2 = 0.41$, thấp hơn Linear Regression. Điều này có thể do dữ liệu huấn luyện hạn chế (chỉ 56 mẫu sau khi bỏ các hàng có giá trị lag thiếu), không đủ để phát huy hết ưu điểm của ensemble method.

2.7.3 5.3 XGBoost Regressor

Giới thiệu thuật toán:

XGBoost (Extreme Gradient Boosting) là thuật toán boosting mạnh mẽ, xây dựng các cây quyết định tuần tự, mỗi cây mới tập trung sửa lỗi của các cây trước đó. XGBoost được biết đến với tốc độ nhanh, khả năng xử lý dữ liệu lớn, và thường cho kết quả xuất sắc trong các cuộc thi khoa học dữ liệu.

Ứng dụng trong bài toán:

XGBoost được kỳ vọng sẽ nắm bắt tốt hơn các mẫu hình phức tạp trong dữ liệu du lịch, đặc biệt là tính mùa vụ và tác động của COVID-19.

Kết quả:

Chỉ số	Giá trị
MAE	1,037,611
RMSE	1,467,481
R^2	0.4709

XGBoost đạt $R^2 = 0.47$, tương đương với Linear Regression. Mặc dù XGBoost là thuật toán mạnh hơn, nhưng với kích thước mẫu nhỏ, ưu điểm của nó không được phát huy đầy đủ.

2.7.4 5.4 SARIMA (Mô hình chuỗi thời gian)

Giới thiệu thuật toán:

SARIMA (Seasonal AutoRegressive Integrated Moving Average) là mô hình chuỗi thời gian classic, phù hợp với dữ liệu có tính mùa vụ. Mô hình kết hợp các thành phần: tự hồi quy (AR), lấy sai phân (I), trung bình trượt (MA), và mùa vụ (S).

Ứng dụng trong bài toán:

SARIMA được áp dụng trên chuỗi thời gian tổng hợp (tổng lượng khách mỗi quý), với tham số mùa vụ = 4 (4 quý/năm). Đặc biệt, năm 2021 (không có dữ liệu) được loại bỏ khỏi chuỗi để tránh ảnh hưởng đến mô hình.

Kết quả:

Chỉ số	Giá trị
MAE	1,548,896
RMSE	2,096,619
R^2	-0.0799

SARIMA cho kết quả kém nhất với R^2 âm (-0.08), nghĩa là mô hình dự đoán còn tệ hơn so với việc dự đoán bằng giá trị trung bình. Nguyên nhân chính là do khoảng trống dữ liệu năm 2021 phá vỡ tính liên tục của chuỗi thời gian, khiến mô hình không thể học được đúng pattern mùa vụ.

2.7.5 5.5 Chronos-T5 (Foundation Model)

Giới thiệu thuật toán:

Chronos là một mô hình foundation model (mô hình nền tảng) cho chuỗi thời gian, được phát triển bởi Amazon. Khác với các mô hình truyền thống (Linear Regression, Random Forest, XGBoost, SARIMA) — vốn chỉ được huấn luyện trên tập dữ liệu nhỏ của bài toán — Chronos đã được tiền huấn luyện (pretrained) trên hàng triệu chuỗi thời gian từ nhiều lĩnh vực khác nhau (tài chính, thời tiết, bán lẻ, giao thông, v.v.).

Chronos sử dụng kiến trúc Transformer (T5) và hoạt động theo nguyên lý **zero-shot forecasting**: không cần huấn luyện lại trên dữ liệu cụ thể của bài toán, chỉ cần cung cấp chuỗi lịch sử (context) là mô hình có thể dự đoán. Điều này đặc biệt hữu ích khi dữ liệu huấn luyện hạn chế — như trong trường hợp của chúng ta chỉ có 55 quý dữ liệu.

Mô hình cũng cung cấp dự đoán xác suất (probabilistic forecast), cho phép tính khoảng tin cậy thay vì chỉ dự đoán điểm.

Ứng dụng trong bài toán:

Chúng ta thử nghiệm 4 kích thước mô hình: Tiny (8M tham số), Small (46M), Base (200M), và Large (710M). Tất cả chạy trên CPU (không có GPU). Toàn bộ 55 quý dữ liệu huấn luyện được đưa vào làm context, và mô hình dự đoán 10 quý tiếp theo (2024 Q1 – 2026 Q2) mà không cần bất kỳ quá trình huấn luyện nào.

Kết quả:

Mô hình	MAE	RMSE	R ²
chronos-t5-tiny	1,018,322	1,297,288	-1.03
chronos-t5-small	941,644	1,228,935	-0.82
chronos-t5-base	919,813	1,158,728	-0.62
chronos-t5-large	1,013,397	1,309,193	-1.07

Nhận xét thú vị: Chronos-t5-base cho MAE thấp nhất trong tất cả các mô hình (919K), thấp hơn cả Linear Regression (1.1M), XGBoost (1.04M). Tuy nhiên, R² của Chronos là âm (-0.62). Điều này có vẻ mâu thuẫn, nhưng thực tế dễ hiểu:

- **MAE thấp** vì Chronos dự đoán trong một dải hẹp quanh mức 3.8–4.4 triệu, nên sai số tuyệt đối trung bình không quá lớn.
- **R² âm** vì Chronos không nắm bắt được sự biến động mạnh của dữ liệu thực tế (từ 3.8 triệu đến 6.8 triệu). R² âm nghĩa là dự đoán bằng giá trị trung bình còn tốt hơn.

Hiện tượng này cũng xảy ra với SARIMA và là dấu hiệu cho thấy **bài toán dự đoán du lịch hậu COVID-19 là cực kỳ khó khăn** đối với mọi mô hình.

Tại sao Base tốt hơn Large? Điều đáng chú ý là chronos-t5-base (200M tham số) cho kết quả tốt hơn chronos-t5-large (710M tham số). Đây không phải là lỗi — nguyên nhân bao gồm:

1. **Ngữ cảnh quá ngắn:** Với chỉ 55 điểm dữ liệu đầu vào, mô hình lớn hơn có xu hướng “overthinking” — tìm kiếm các mẫu hình phức tạp trong khi thực tế chỉ có một xu hướng tăng đơn giản bị gián đoạn bởi COVID-19.
2. **Prior quá mạnh:** Mô hình lớn hơn đã học được nhiều pattern hơn từ dữ liệu tiền huấn luyện, nhưng các pattern đó (từ tài chính, thời tiết, bán lẻ...) có thể không phù hợp với dữ liệu du lịch Việt Nam. Mô hình base “ít kiến thức hơn” nên ít bị ảnh hưởng bởi prior sai.
3. **Hiệu ứng overfitting zero-shot:** Trong zero-shot forecasting, lớn hơn không phải lúc nào cũng tốt hơn. Điều này đã được ghi nhận trong nhiều benchmark gần đây (QuitoBench 2026, TIME benchmark).

Đây là một bài viết quan trọng: **khi chọn foundation model, cần benchmark trên dữ liệu thực tế thay vì mặc định chọn model lớn nhất.**

2.7.6 5.6 So sánh hiệu suất các mô hình

```
[ ]: # Run models and create comparison
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import xgboost as xgb
from statsmodels.tsa.statespace.sarimax import SARIMAX

quarter_map = {'Q1':1, 'Q2':2, 'Q3':3, 'Q4':4}
df_long['quarter_num'] = df_long['quarter'].map(quarter_map)
df_long['time_idx'] = df_long['year'] + (df_long['quarter_num']-1)/4
df_long = df_long.sort_values(['country', 'year', 'quarter_num'])
for lag in [1,2,4]:
    df_long[f'lag_{lag}'] = df_long.groupby('country')['arrivals'].shift(lag)
df_long['rolling_mean_4'] = df_long.groupby('country')['arrivals'].
    ↪transform(lambda x: x.rolling(4, min_periods=1).mean())

agg_all = df_long.groupby(['year', 'quarter_num', 'time_idx'])['arrivals'].sum().
    ↪reset_index().sort_values(['year', 'quarter_num'])
agg_all['lag_1'] = agg_all['arrivals'].shift(1)
agg_all['lag_4'] = agg_all['arrivals'].shift(4)
agg_all['rolling_mean_4'] = agg_all['arrivals'].rolling(4, min_periods=1).mean()
feat = ['year', 'quarter_num', 'time_idx', 'lag_1', 'lag_4', 'rolling_mean_4']
tr = agg_all[agg_all['year']<=2023].dropna(subset=['lag_1', 'lag_4'])
te = agg_all[agg_all['year']>=2024].dropna(subset=['lag_1', 'lag_4'])
X_train, y_train = tr[feat].values, tr['arrivals'].values
X_test, y_test = te[feat].values, te['arrivals'].values

lr = LinearRegression().fit(X_train, y_train)
y_lr = lr.predict(X_test)
rf = RandomForestRegressor(n_estimators=200, max_depth=10, random_state=42).
    ↪fit(X_train, y_train)
y_rf = rf.predict(X_test)
xgb_m = xgb.XGBRegressor(n_estimators=200, max_depth=6, learning_rate=0.1,
    ↪random_state=42).fit(X_train, y_train)
y_xgb = xgb_m.predict(X_test)

agg_ts = df_long[df_long['year']<=2023].
    ↪groupby(['year', 'quarter_num'])['arrivals'].sum().reset_index().
    ↪sort_values(['year', 'quarter_num'])
agg_ts = agg_ts[agg_ts['year']!=2021]
sarima = SARIMAX(agg_ts['arrivals'].values, order=(1,1,1),
    ↪seasonal_order=(1,1,1,4), enforce_stationarity=False,
    ↪enforce_invertibility=False).fit(dispatch=False, maxiter=500)
```

```

test_ts = df_long[df_long['year']>=2024].
    ↳groupby(['year', 'quarter_num'])['arrivals'].sum().reset_index().
    ↳sort_values(['year', 'quarter_num'])
sp = sarima.forecast(steps=len(test_ts))
y_s = test_ts['arrivals'].values[:len(sp)]

# Chronos zero-shot forecast
import torch
from chronos import ChronosPipeline
train_agg = df_long[df_long['year']<=2023].
    ↳groupby(['year', 'quarter_num'])['arrivals'].sum().reset_index().
    ↳sort_values(['year', 'quarter_num'])
train_agg = train_agg[train_agg['year']!=2021]
context = torch.tensor(train_agg['arrivals'].values, dtype=torch.float32).
    ↳unsqueeze(0)
chronos_pipe = ChronosPipeline.from_pretrained('amazon/chronos-t5-base',
    ↳device_map='cpu', dtype=torch.float32)
chronos_fc = chronos_pipe.predict(context, prediction_length=len(test_ts))
y_chronos = np.median(chronos_fc[0].numpy(), axis=0)

mae_lr = mean_absolute_error(y_test, y_lr); rmse_lr = np.
    ↳sqrt(mean_squared_error(y_test, y_lr)); r2_lr = r2_score(y_test, y_lr)
mae_rf = mean_absolute_error(y_test, y_rf); rmse_rf = np.
    ↳sqrt(mean_squared_error(y_test, y_rf)); r2_rf = r2_score(y_test, y_rf)
mae_xgb = mean_absolute_error(y_test, y_xgb); rmse_xgb = np.
    ↳sqrt(mean_squared_error(y_test, y_xgb)); r2_xgb = r2_score(y_test, y_xgb)
mae_s = mean_absolute_error(y_s, sp); rmse_s = np.sqrt(mean_squared_error(y_s,
    ↳sp)); r2_s = r2_score(y_s, sp)
mae_c = mean_absolute_error(y_test, y_chronos); rmse_c = np.
    ↳sqrt(mean_squared_error(y_test, y_chronos)); r2_c = r2_score(y_test,
    ↳y_chronos)

comparison = pd.DataFrame({'Model': ['Linear Regression', 'Random
    ↳Forest', 'XGBoost', 'SARIMA', 'Chronos-T5-Base'],
    'MAE': [mae_lr, mae_rf, mae_xgb, mae_s, mae_c], 'RMSE':
    ↳[rmse_lr, rmse_rf, rmse_xgb, rmse_s, rmse_c], 'R²':
    ↳[r2_lr, r2_rf, r2_xgb, r2_s, r2_c]})
print(comparison.to_string(index=False))

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
models = comparison['Model']; x = range(len(models))
for ax, metric, title in zip(axes, ['MAE', 'RMSE', 'R²'], ['MAE', 'RMSE', 'R²
    ↳Score']):
    vals = comparison[metric]/1e6 if metric != 'R²' else comparison[metric]
    ax.bar(x, vals, color=sns.color_palette('Set2', len(models)))

```

```

    ax.set_xticks(x); ax.set_xticklabels(models, rotation=45, ha='right',
↪ fontsize=9)
    ax.set_title(title, fontweight='bold')
    if metric == 'R2': ax.axhline(y=0, color='red', ls='--', alpha=0.5)
plt.suptitle('Hình 6: So sánh hiệu suất các mô hình dự đoán', fontweight='bold')
plt.tight_layout(); plt.savefig('output/model_comparison.png', dpi=150,
↪ bbox_inches='tight'); plt.show()

```

```

[ ]: results = te[['year', 'quarter_num']].copy()
results['quarter'] = 'Q' + results['quarter_num'].astype(str)
results['actual'] = y_test
results['LR'] = y_lr; results['RF'] = y_rf; results['XGB'] = y_xgb;
↪ results['Chronos'] = y_chronos
results['LR_err%'] = ((y_lr - y_test) / y_test * 100).round(1)
results['RF_err%'] = ((y_rf - y_test) / y_test * 100).round(1)
results['XGB_err%'] = ((y_xgb - y_test) / y_test * 100).round(1)
results['Chr_err%'] = ((y_chronos - y_test) / y_test * 100).round(1)
print(results[['year', 'quarter', 'actual', 'LR', 'LR_err%', 'XGB', 'XGB_err%', 'Chronos', 'Chr_err%']
↪ to_string(index=False))

fig, ax = plt.subplots(figsize=(14, 6))
x = range(len(results)); labels = results['year'].astype(str) + ' ' +
↪ results['quarter']
ax.bar([i-0.3 for i in x], results['actual']/1e6, 0.2, label='Thực tế',
↪ color='#2196F3')
ax.bar([i-0.1 for i in x], results['LR']/1e6, 0.2, label='LR', color='#FF9800')
ax.bar([i+0.1 for i in x], results['XGB']/1e6, 0.2, label='XGB',
↪ color='#4CAF50')
ax.bar([i+0.3 for i in x], results['Chronos']/1e6, 0.2, label='Chronos',
↪ color='#E91E63')
ax.set_xticks(x); ax.set_xticklabels(labels, rotation=45, ha='right')
ax.set_ylabel('Lượng khách (triệu)'); ax.set_title('Hình 6b: Dự đoán vs Thực tế
↪ Tập test (2024-2026)', fontweight='bold')
ax.legend(); ax.grid(axis='y', alpha=0.3)
plt.tight_layout(); plt.savefig('output/pred_vs_actual.png', dpi=150,
↪ bbox_inches='tight'); plt.show()

```

Nhận xét chi tiết theo quý:

Bảng trên cho thấy chi tiết dự đoán so với thực tế cho từng quý trong tập test:

- **Quý 2024 Q3** là dự đoán chính xác nhất: XGBoost chỉ sai lệch -1.2%, Chronos sai +2.4%.
- **Quý 2025 Q1 và 2026 Q1** là dự đoán kém nhất: tất cả các mô hình đều dự đoán thấp hơn đáng kể (sai lệch -23% đến -39%).
- **Xu hướng chung:** Tất cả các mô hình đều có xu hướng **underestimate** (dự đoán thấp hơn thực tế), đặc biệt với các quý có lượng khách cao.

Thách thức “hậu COVID-19” (Post-COVID Challenge):

Kết quả trên chỉ ra một vấn đề cốt lõi: **không mô hình nào — kể cả Chronos foundation model được pretrained trên hàng triệu chuỗi thời gian — có thể dự đoán chính xác sự phục hồi hậu đại dịch**. Đây là một hiện tượng phổ biến trong dự đoán chuỗi thời gian sau các sự kiện bất khả kháng (black swan events):

1. **Dữ liệu huấn luyện kết thúc năm 2023** với mức ~5 triệu lượt/quý, nhưng năm 2025-2026 bùng nổ lên 6-7 triệu. Mô hình chỉ dựa vào lịch sử không thể “biết” rằng ngành du lịch sẽ tăng trưởng vượt bậc.
2. **Foundation model cũng không ngoại lệ:** Chronos-T5-Base (200M tham số, pretrained trên hàng triệu chuỗi) cho MAE thấp nhất (920K) nhưng R^2 âm (-0.62). Mô hình dự đoán một dải hẹp quanh 3.8-4.4 triệu, bỏ lỡ hoàn toàn sự biến động thực tế.
3. **SARIMA bị ảnh hưởng nặng nhất** do khoảng trống dữ liệu 2021 phá vỡ tính liên tục của chuỗi thời gian.
4. **Linear Regression** paradoxically cho R^2 tốt nhất (0.48) vì nó nắm bắt được xu hướng tăng tuyến tính, dù sai số tuyệt đối vẫn lớn.

Bài học: Dự đoán thời gian thực sau đại dịch đòi hỏi dữ liệu bên ngoài (leading indicators) như chính sách visa, tỷ giá, giá vé máy bay, sự kiện du lịch — không chỉ dựa vào dữ liệu lịch sử.

Nhận xét tổng thể:

Mô hình	MAE	RMSE	R^2	Ưu điểm	Nhược điểm
Linear Regression	1.10M	1.45M	0.48	R^2 tốt nhất, đơn giản	Sai số tuyệt đối lớn
Random Forest	1.10M	1.55M	0.41	Linh hoạt	Dữ liệu nhỏ quá ít
XGBoost	1.04M	1.47M	0.47	MAE thấp (ML models)	Cần nhiều dữ liệu hơn
SARIMA	1.55M	2.10M	-0.08	Mùa vụ rõ ràng	Phá vỡ bởi gap 2021
Chronos-T5-Base	0.92M	1.16M	-0.62	MAE thấp nhất, zero-shot	Không nắm bắt biến động

Chronos-T5-Base cho MAE thấp nhất trong tất cả các mô hình (920K), chứng tỏ sức mạnh của foundation model. Tuy nhiên, R^2 âm cho thấy nó dự đoán quá “an toàn” — một dải hẹp quanh mức trung bình thay vì nắm bắt sự biến động thực tế. Ngược lại, **Linear Regression** cho R^2 tốt nhất (0.48) vì nó học được xu hướng tăng tuyến tính, dù sai số tuyệt đối lớn hơn.

Với kích thước mẫu nhỏ (chỉ 55 quý huấn luyện), các mô hình phức tạp hơn (Random Forest, XGBoost) không có lợi thế đáng kể so với Linear Regression đơn giản. Đây là một insight quan trọng: **với dữ liệu nhỏ, mô hình đơn giản thường tốt hơn mô hình phức tạp**. Và với dữ liệu có structural break (như COVID-19), **không mô hình nào dự đoán tốt được nếu chỉ dựa vào dữ liệu lịch sử**.

2.8 6. Tối ưu mô hình

Để cải thiện hiệu suất của các mô hình, nhóm thực hiện tối ưu hóa siêu tham số (hyperparameter tuning) cho Random Forest và XGBoost.

2.8.1 6.1 GridSearchCV cho Random Forest

Khái niệm:

GridSearchCV là kỹ thuật tìm kiếm lưới (grid search) kết hợp với cross-validation. Phương pháp này thử tất cả các tổ hợp có thể của các siêu tham số được chỉ định, và chọn tổ hợp cho điểm cross-validation tốt nhất.

Cấu hình thuật toán:

Các siêu tham số được tối ưu cho Random Forest: - `n_estimators`: Số lượng cây (100, 200, 300) - `max_depth`: Độ sâu tối đa (5, 10, 15, None) - `min_samples_split`: Số mẫu tối thiểu để chia nút (2, 5, 10)

Sử dụng 3-fold cross-validation và đánh giá bằng MAE (Mean Absolute Error).

Kết quả:

Siêu tham số tốt nhất: {'max_depth': 5, 'min_samples_split': 2, 'n_estimators': 200}

Chỉ số	Trước tối ưu	Sau tối ưu	Cải thiện
MAE	1,098,133	1,081,460	-1.5%
RMSE	1,548,464	1,538,767	-0.6%
R ²	0.4109	0.4183	+1.8%

2.8.2 6.2 RandomizedSearchCV cho XGBoost

Khái niệm:

RandomizedSearchCV tương tự GridSearchCV, nhưng thay vì thử tất cả các tổ hợp, nó chỉ thử một số lượng ngẫu nhiên nhất định các tổ hợp. Phương pháp này nhanh hơn grid search khi không gian siêu tham số lớn.

Cấu hình thuật toán:

Các siêu tham số được tối ưu cho XGBoost: - `n_estimators`: 100, 200, 300, 500 - `max_depth`: 3, 5, 7, 9 - `learning_rate`: 0.01, 0.05, 0.1, 0.2 - `subsample`: 0.7, 0.8, 0.9, 1.0 - `colsample_bytree`: 0.7, 0.8, 0.9, 1.0

Sử dụng 50 lần thử ngẫu nhiên, 3-fold cross-validation.

Kết quả:

Siêu tham số tốt nhất: {'subsample': 0.7, 'n_estimators': 200, 'max_depth': 9, 'learning_rate': 0.2, 'colsample_bytree': 0.7}

Chỉ số	Trước tối ưu	Sau tối ưu	Cải thiện
MAE	1,037,611	1,092,469	+5.3% (xấu hơn)
RMSE	1,467,481	1,553,966	+5.9% (xấu hơn)
R ²	0.4709	0.4067	-13.6% (xấu hơn)

2.8.3 6.3 Kết quả sau tối ưu

Điều thú vị là tối ưu hóa siêu tham số không cải thiện hiệu suất cho XGBoost — thực tế, kết quả còn **xấu hơn** so với tham số mặc định. Hiện tượng này được gọi là “**overfitting trên cross-validation**”: mô hình tối ưu quá mức trên tập huấn luyện (thông qua cross-validation) nhưng lại hoạt động kém hơn trên tập kiểm tra.

Đối với Random Forest, tối ưu hóa chỉ cải thiện nhẹ (R^2 tăng từ 0.41 lên 0.42).

Bảng so sánh cuối cùng:

```
[ ]: from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
rf_gs = GridSearchCV(RandomForestRegressor(random_state=42), {'n_estimators':
    ↪ [100,200,300], 'max_depth': [5,10,15,None], 'min_samples_split': [2,5,10]},
    ↪ cv=3, scoring='neg_mean_absolute_error', n_jobs=-1).fit(X_train, y_train)
y_rf_gs = rf_gs.predict(X_test)
xgb_rs = RandomizedSearchCV(xgb.XGBRegressor(random_state=42), {'n_estimators':
    ↪ [100,200,300,500], 'max_depth': [3,5,7,9], 'learning_rate': [0.01,0.05,0.1,0.
    ↪ 2], 'subsample': [0.7,0.8,0.9,1.0], 'colsample_bytree': [0.7,0.8,0.9,1.0]},
    ↪ n_iter=50, cv=3, scoring='neg_mean_absolute_error', n_jobs=-1,
    ↪ random_state=42).fit(X_train, y_train)
y_xgb_rs = xgb_rs.predict(X_test)
mae_rf_gs = mean_absolute_error(y_test, y_rf_gs); rmse_rf_gs = np.
    ↪ sqrt(mean_squared_error(y_test, y_rf_gs)); r2_rf_gs = r2_score(y_test,
    ↪ y_rf_gs)
mae_xgb_rs = mean_absolute_error(y_test, y_xgb_rs); rmse_xgb_rs = np.
    ↪ sqrt(mean_squared_error(y_test, y_xgb_rs)); r2_xgb_rs = r2_score(y_test,
    ↪ y_xgb_rs)

comp = pd.DataFrame({'Model': ['Linear Regression', 'Random
    ↪ Forest', 'XGBoost', 'SARIMA', 'RF (optimized)', 'XGBoost (optimized)'], 'MAE':
    ↪ [mae_lr,mae_rf,mae_xgb,mae_s,mae_rf_gs,mae_xgb_rs], 'RMSE':
    ↪ [rmse_lr,rmse_rf,rmse_xgb,rmse_s,rmse_rf_gs,rmse_xgb_rs], 'R²':
    ↪ [r2_lr,r2_rf,r2_xgb,r2_s,r2_rf_gs,r2_xgb_rs]})
print(comp.to_string(index=False))
```

2.9 7. Dự đoán tương lai

2.9.1 7.1 Dự đoán 4 quý tiếp theo bằng SARIMA

Mặc dù SARIMA cho kết quả kém trên tập kiểm tra, nhóm vẫn sử dụng mô hình này để dự đoán 4 quý tiếp theo vì SARIMA là mô hình chuỗi thời gian chuyên dụng cho dự đoán, và nó có thể cung cấp khoảng tin cậy (confidence interval) — một tính năng quan trọng cho việc ra quyết định.

Mô hình SARIMA được huấn luyện lại trên toàn bộ dữ liệu (loại bỏ năm 2021) trước khi dự đoán.

```
[ ]: full_ts = df_long.groupby(['year', 'quarter_num'])['arrivals'].sum().
    ↪ reset_index().sort_values(['year', 'quarter_num'])
full_ts = full_ts[full_ts['year']!=2021]
```

```

s_full = SARIMAX(full_ts['arrivals'].values, order=(1,1,1),
    ↪seasonal_order=(1,1,1,4), enforce_stationarity=False,
    ↪enforce_invertibility=False).fit(dispatch=False, maxiter=500)
fc = s_full.get_forecast(steps=4)
fc_mean = np.array(fc.predicted_mean).flatten()
fc_ci = np.array(fc.conf_int(alpha=0.05))
ly = full_ts['year'].max(); lq = full_ts[full_ts['year']==ly]['quarter_num'].
    ↪max()
fqs = []; y, q = ly, lq
for _ in range(4):
    q += 1
    if q > 4: q = 1; y += 1
    fqs.append((y, q))
print("Dự đoán SARIMA cho 4 quý tiếp theo:")
for (yr, qr), v, lo, hi in zip(fqs, fc_mean, fc_ci[:,0], fc_ci[:,1]):
    print(f" {yr} Q{qr}: {v:,.0f} [{lo:,.0f} - {hi:,.0f}]")

fig, ax = plt.subplots(figsize=(14, 7))
hist = full_ts.copy(); hist['lbl'] = hist['year'].
    ↪astype(str)+'Q'+hist['quarter_num'].astype(str)
ax.plot(range(len(hist)), hist['arrivals']/1e6, 'o-', lw=2, ms=5,
    ↪color='#2196F3', label='Dữ liệu lịch sử')
fcx = range(len(hist), len(hist)+4)
ax.plot(fcx, fc_mean/1e6, 's-', lw=2, ms=8, color='#FF5722', label='Dự đoán')
ax.fill_between(fcx, fc_ci[:,0]/1e6, fc_ci[:,1]/1e6, alpha=0.3,
    ↪color='#FF5722', label='Khoảng tin cậy 95%')
ax.axvline(x=len(hist)-0.5, color='gray', ls='--', alpha=0.5)
ax.set_xlabel('Quý'); ax.set_ylabel('Tổng lượng khách (triệu)')
ax.set_title('Hình 7: Dự đoán lượng khách 4 quý tiếp theo với khoảng tin cậy
    ↪95%', fontweight='bold')
ax.legend(fontsize=11); ax.grid(True, alpha=0.3)
plt.tight_layout(); plt.savefig('output/forecast_plot.png', dpi=150,
    ↪bbox_inches='tight'); plt.show()

```

2.9.2 7.2 Phân tích khoảng tin cậy

Nhận xét về dự đoán:

Kết quả dự đoán cho thấy một số vấn đề cần lưu ý:

- Dự đoán cho Q1/2027 là dương (~1.4 triệu), nhưng các quý tiếp theo cho giá trị âm hoặc gần 0. Điều này phản ánh mô hình SARIMA không phù hợp với dữ liệu có khoảng trống lớn (năm 2021).
- **Khoảng tin cậy 95% rất rộng**, cho thấy mức độ không chắc chắn cao. Điều này là hợp lý vì:
 - Dữ liệu bị gián đoạn bởi COVID-19
 - Chỉ có 56 mẫu huấn luyện (sau khi loại bỏ 2021)

– Lượng khách có tính biến động cao

Lưu ý quan trọng: Các dự đoán này chỉ mang tính tham khảo và không nên được sử dụng làm cơ sở duy nhất cho việc ra quyết định. Cần kết hợp với phân tích định tính và các yếu tố bên ngoài (chính sách visa, tình hình kinh tế, etc.) để đưa ra dự đoán chính xác hơn.

2.10 8. Tổng kết

2.10.1 8.1 Kết quả đạt được

Đề tài đã hoàn thành các mục tiêu đề ra:

1. **Phân tích dữ liệu thành công:** Đã phân tích và trực quan hóa dữ liệu lượng khách quốc tế đến Việt Nam giai đoạn 2008–2026, khám phá các xu hướng tăng trưởng, tác động của COVID-19, và sự phục hồi mạnh mẽ của ngành du lịch.
2. **Xác định thị trường nguồn quan trọng:** Trung Quốc và Hàn Quốc là hai thị trường lớn nhất, với Hàn Quốc là thị trường tăng trưởng nhanh nhất và phục hồi sau COVID-19 tốt nhất.
3. **Phát hiện tính mùa vụ:** Quý 1 có lượng khách trung bình cao nhất, phản ánh mùa du lịch đầu năm.
4. **Xây dựng mô hình dự đoán:** Đã xây dựng và so sánh 4 mô hình. Linear Regression cho kết quả tốt nhất ($R^2 = 0.48$), cho thấy mối quan hệ tuyến tính giữa các đặc trưng và lượng khách là đáng kể.
5. **Tối ưu hóa siêu tham số:** Đã áp dụng GridSearchCV và RandomizedSearchCV, rút ra bài viết rằng tối ưu hóa không phải lúc nào cũng cải thiện hiệu suất, đặc biệt với dữ liệu nhỏ.

2.10.2 8.2 Hạn chế

Đề tài có một số hạn chế:

1. **Dữ liệu hạn chế:** Chỉ có 56 mẫu huấn luyện (sau khi tạo lag features), không đủ để phát huy hết ưu điểm của các mô hình phức tạp.
2. **Khoảng trống dữ liệu:** Năm 2021 không có dữ liệu, phá vỡ tính liên tục của chuỗi thời gian, ảnh hưởng nghiêm trọng đến hiệu suất của SARIMA.
3. **Thiếu đặc trưng bên ngoài:** Mô hình chỉ sử dụng dữ liệu lịch sử, chưa kết hợp các yếu tố bên ngoài như chính sách visa, tỷ giá, sự kiện đặc biệt, etc.
4. **Dự đoán SARIMA không ổn định:** Kết quả dự đoán cho thấy giá trị âm hoặc gần 0, phản ánh mô hình không phù hợp với dữ liệu có khoảng trống.

2.10.3 8.3 Hướng phát triển

Để cải thiện đề tài trong tương lai:

1. **Thu thập thêm dữ liệu:** Bổ sung dữ liệu theo tháng (thay vì theo quý) để tăng kích thước mẫu.
2. **Thêm đặc trưng bên ngoài:** Kết hợp dữ liệu về chính sách visa, tỷ giá hối đoái, sự kiện du lịch, và tình hình kinh tế các quốc gia nguồn.

3. **Thử nghiệm các mô hình khác:** ARIMA với xử lý khoảng trống tốt hơn, Prophet (Facebook), hoặc các mô hình deep learning như LSTM.
4. **Dự đoán theo từng quốc gia:** Xây dựng mô hình riêng cho từng thị trường nguồn để có dự đoán chi tiết hơn.
5. **Xây dựng dashboard tương tác:** Tạo dashboard để theo dõi và dự đoán lượng khách theo thời gian thực.

2.11 Tài liệu tham khảo

1. Scikit-learn documentation: <https://scikit-learn.org/>
2. XGBoost documentation: <https://xgboost.readthedocs.io/>
3. Statsmodels SARIMA documentation: <https://www.statsmodels.org/>
4. Pandas documentation: <https://pandas.pydata.org/>
5. Matplotlib documentation: <https://matplotlib.org/>